SmartCart ADMIN Role Guide

Oct 10, 2026 · @Abdelrahman Ashraf

Overview

New accounts will wait in **Pending approval** until an admin approves them, and only admins will be able to deactivate, reactivate or delete accounts. Half of the groundwork already exists: `ROLE_ADMIN` is seeded by `RoleSeeder`, `@EnableMethodSecurity` is on in `SecurityConfig`, and `JwtFilter` reloads roles from the database on every request.

What changes for each kind of user:

Who	Today	After this guide
New customer (email sign-up)	Gets a token and is signed in at once	Account is created as Pending approval; no token until approved
New customer (Google)	Signed in at once	Same as above: pending until approved
Existing customer	Can deactivate or delete their own account	Can only ask; the action is done by an admin
Deactivated customer	Reactivates themselves via `POST /auth/reactivate`	An admin reactivates them
Admin	Does not exist yet	Approves, rejects, deactivates, reactivates and deletes accounts, and manages the catalogue

The account lifecycle is drawn in Step 2. Every rule is enforced in the backend; the Angular guards only hide screens.

Explained like you're 5

Think of the shop as a building with doors. Everyone who comes in gets a **badge** (the JWT). On the badge is a **sticker** that says what you are: `ROLE_CUSTOMER` or `ROLE_ADMIN`.

- **The guard at the front door** is `SecurityConfig`. It decides which hallways you may walk into, for example "only badges with an ADMIN sticker may enter `/api/v1/admin/**`".

- **The guard at a single room** is `@PreAuthorize("hasRole('ADMIN')")` on a method. Even if you sneak past the hallway, this room checks your sticker again.
- **The sticker is checked against the office list every time.** Your `JwtFilter` reloads the user from the database on each request, so if an admin removes someone's sticker, it stops working at once, even while their old badge is still valid.
- **A new visitor waits in the lobby** (Pending approval) until an admin says "let them in". Waiting in the lobby means Spring treats the account as *not enabled*, so the login door stays shut.

One naming rule to remember: the database stores `ROLE_ADMIN`, and `hasRole('ADMIN')` adds the `ROLE_` prefix for you. `hasAuthority('ROLE_ADMIN')` is the same check written out in full.

Step 1: Create the first admin

The ADMIN role already exists, so the only missing piece is an admin *account*. Create it once at startup from environment variables, never from a sign-up form and never with a password written in code.

1. Add the settings to `application.properties`, with no default for the password:

```
# First admin, created only when no admin exists yet
app.admin.email=${ADMIN_EMAIL:}
app.admin.password=${ADMIN_PASSWORD:}
app.admin.name=${ADMIN_NAME:Store Admin}
```

2. Add a query to `CustomerRepo` that asks "is there any admin?". Spring Data can walk into the `roles` list by name:

```
boolean existsByRolesName(String roleName);
```

3. Create `config/AdminSeeder.java`. Give it `@Order(2)` and `RoleSeeder` `@Order(1)`, because the role must exist first:

```
@Slf4j
@Component
@Order(2)
@RequiredArgsConstructor
public class AdminSeeder implements ApplicationRunner
{
    private final CustomerRepo customerRepo;
    private final RoleRepo roleRepo;
    private final PasswordEncoder passwordEncoder;
```

```
    @Value("${app.admin.email}") private String email;
    @Value("${app.admin.password}") private String password;
    @Value("${app.admin.name}") private String name;

    @Override
    @Transactional
    public void run(ApplicationArguments args)
    {
        // Runs on every startup: only act when the shop has no admin at all.
        if (customerRepo.existsByRolesName(Role.ADMIN))
            return;
        if (StringUtils.isAnyBlank(email, password))
        {
            log.warn("No admin exists. Set ADMIN_EMAIL and ADMIN_PASSWORD to
create one.");
            return;
        }

        String normalizedEmail = email.trim().toLowerCase(Locale.ROOT);
        Role adminRole = roleRepo.findByName(Role.ADMIN).orElseThrow();
        Role customerRole = roleRepo.findByName(Role.CUSTOMER).orElseThrow();

        // EMAIL is unique: inserting a second row for an email a customer
already uses would fail at startup.
        // Promote that account instead. It keeps its own password;
ADMIN_PASSWORD is not applied to it.
        Optional<Customer> existing =
customerRepo.findByEmailIgnoreCase(normalizedEmail);
        if (existing.isPresent())
        {
            Customer customer = existing.get();
            customer.getRoles().add(adminRole);
            customer.setStatus(AccountStatus.ACTIVE); // added in Step 2
            log.info("Promoted existing account {} to admin",
normalizedEmail);
            return;
        }

        Customer admin = new Customer();
        admin.setName(name);
        admin.setEmail(normalizedEmail);
        admin.setPassword(passwordEncoder.encode(password));
        admin.setAuthProvider(AuthProvider.LOCAL);
        admin.setStatus(AccountStatus.ACTIVE); // added in Step 2
        admin.setRoles(new ArrayList<>(List.of(adminRole, customerRole)));
```

```
            customerRepo.save(admin);
            log.info("Created the first admin account {}", normalizedEmail);
        }
    }
```

The check is by role, not by email, because the goal is "never zero admins". Changing `ADMIN_EMAIL` later therefore does not create a second admin. If the email already belongs to a customer, that account is promoted instead of a duplicate being inserted. `findByEmailIgnoreCase` already loads roles through its `@EntityGraph`, so `getRoles().add(...)` works without extra queries.

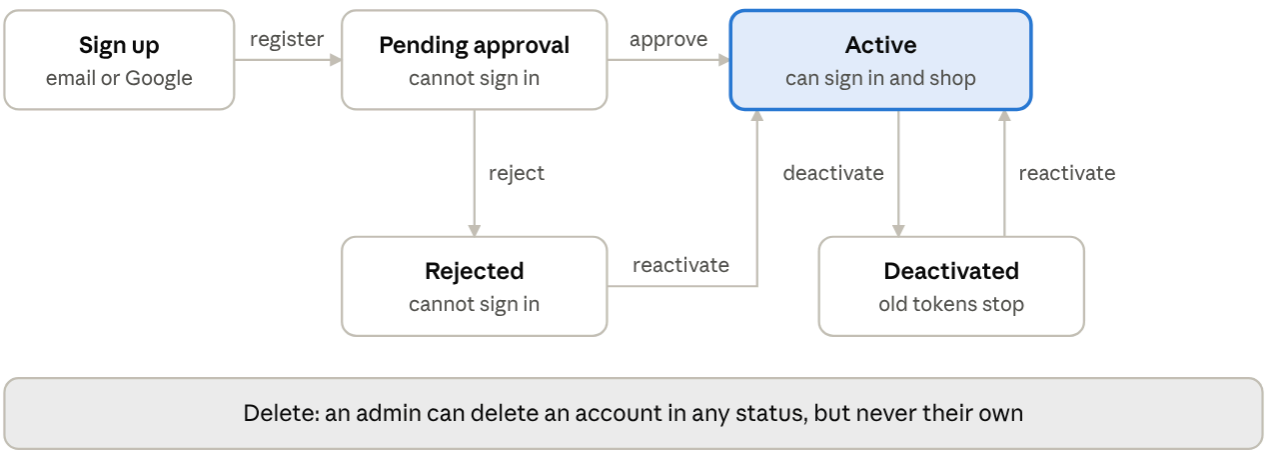
4. Put `ADMIN_EMAIL` and `ADMIN_PASSWORD` in your git-ignored `.env` and in `docker-compose.yml` as `${ADMIN_PASSWORD}`, not as a literal value.

Why give the admin both roles: they can still shop and test the customer screens with the same account. Promoting another user later is an admin feature (see the ideas section), not a database edit.

Step 2: Add an account status

Replace the `enabled` true/false on `Customer` with a status that has four values. One boolean cannot say "waiting" and "switched off" at the same time, and the admin screen needs to tell them apart.

```
public enum AccountStatus
{
    PENDING_APPROVAL, // signed up, waiting for an admin
    ACTIVE,           // approved, can sign in
    REJECTED,         // an admin said no
    DEACTIVATED       // was active, switched off by an admin
}
```



account lifecycle · 4 statuses, 5 admin actions

Every arrow after sign-up is an admin action; the customer can never move their own account to another status.

In `Customer`, swap the field and keep `isEnabled()` so Spring Security keeps working unchanged:

```
// The DB default only fills rows that existed before this column; new rows
always get the Java value.
@Enumerated(EnumType.STRING)
@ColumnDefault("'ACTIVE'")
@Column(name = "STATUS", nullable = false, length = 20)
private AccountStatus status = AccountStatus.PENDING_APPROVAL;

@Override
public boolean isEnabled()
{
    return this.status == AccountStatus.ACTIVE;
}
```

Because `CustomerUserDetailsService` already filters on `isEnabled()`, a pending, rejected or deactivated user is refused at login **and** loses access on the next request with an old token. You get that for free.

Database migration (do not skip)

`ddl-auto=update` adds the new column but never removes the old one. Two things follow. Postgres refuses to add a `NOT NULL` column to a table that already has rows unless it has a default, which is why `@ColumnDefault("'ACTIVE'")` is there. And the old `IS_ENABLED`

column is also `NOT NULL`, so once the field is gone from the entity every sign-up would fail. Run this once, after the app has started and added `STATUS`:

```
-- Existing rows already got 'ACTIVE' from the default; fix the ones that
were switched off.
UPDATE customer SET status = 'DEACTIVATED' WHERE is_enabled = false;

ALTER TABLE customer DROP COLUMN is_enabled;
```

Existing customers become ACTIVE, so nobody is locked out by the change. Only new sign-ups wait for approval. This is a good moment to switch to Flyway, so changes like this run automatically and in order.

Registration and sign-in changes

1. **`AuthServiceImpl.register`**: save the customer as `PENDING_APPROVAL` and stop calling `issueToken`. Return `202 Accepted` with a small body such as `{"status":"PENDING_APPROVAL"}`. Record `ACCOUNT_REGISTERED` in the audit log as today.
2. **`login`**: in the `DisabledException` branch, after the password check, throw an error that matches the status, so the screen can say something useful.
3. **Google sign-in** (`loginWithOAuth2` and `createOAuth2Customer`): new Google users also start as `PENDING_APPROVAL`. Remove the block that re-enables a deactivated account, and throw the matching error when the status is not `ACTIVE`. `OAuth2LoginFailureHandler` then sends the code to the SPA.
4. **Delete `POST /auth/reactivate`** and remove it from `PUBLIC_AUTH_ENDPOINTS`. Reactivation becomes an admin action.
5. **Remove `PATCH /users/me/deactivate`** and **`DELETE /users/me`** from `CustomerController`. The service methods move to the admin service in Step 3.

New entries for `ErrorCode`:

Code	HTTP	Message
`ACCOUNT_PENDING_APPROVAL`	403	Your account is waiting for approval
`ACCOUNT_REJECTED`	403	Your registration was not approved
`ACCOUNT_NOT_PENDING`	409	Only pending accounts can be approved or rejected
`CANNOT_MODIFY_SELF`	409	Admins cannot deactivate or delete their own account

`ACCOUNT_DEACTIVATED` already exists and keeps its meaning.

Step 3: Admin-only endpoints

Put every admin action under one path, `/api/v1/admin/**`, in its own package (`admin/controller`, `admin/service`, `admin/dtos`). One path means one security rule protects all of it, and nobody can forget to protect a new endpoint.

Lock the hallway and the room

In `SecurityConfig`, add the rule **before** `anyRequest().authenticated()`, because the first matching rule wins:

```
.requestMatchers("/api/v1/admin/**").hasRole("ADMIN")
.anyRequest().authenticated())
```

Then add `@PreAuthorize("hasRole('ADMIN')")` on the admin controller class as a second lock. If someone later moves a method to another path, it stays protected.

The endpoints

Method	Path	What it does	Allowed from
GET	`/api/v1/admin/customers?status=PENDING_APPROVAL&search=`	Paged list, filter by status, search by name or email	any
GET	`/api/v1/admin/customers/{id}`	One customer with roles, status and dates	any
POST	`/api/v1/admin/customers/{id}/approve`	Pending to Active	PENDING_APPROVAL
POST	`/api/v1/admin/customers/{id}/reject`	Pending to Rejected; body `{ "reason": "..." }`	PENDING_APPROVAL
POST	`/api/v1/admin/customers/{id}/deactivate`	Active to Deactivated	ACTIVE
POST	`/api/v1/admin/customers/{id}/reactivate`	Deactivated or Rejected to Active	DEACTIVATED, REJECTED
DELETE	`/api/v1/admin/customers/{id}`	Deletes the account, cart and orders	any except self

Method	Path	What it does	Allowed from
GET	`/api/v1/admin/customers/{id}/activity`	That customer's audit log	any

Use `POST` for the state changes rather than `PATCH status=...`. Each action has its own rule and audit entry, and a named action is harder to misuse.

The service, with the safety rules

Add to `CustomerRepo`:

```
Page<Customer> findByStatus(AccountStatus status, Pageable pageable);
```

Then the admin service. Every method checks the current status first, so approving twice returns a clear 409 instead of silently doing nothing:

```
@Service
@RequiredArgsConstructor
public class AdminCustomerService
{
    private final CustomerRepo customerRepo;
    private final AuditLogService auditLogService;

    @Transactional
    public void approve(UUID customerId)
    {
        Customer customer = find(customerId);
        requireStatus(customer, AccountStatus.PENDING_APPROVAL);
        customer.setStatus(AccountStatus.ACTIVE);
        auditLogService.record(customerId, AuditAction.ACCOUNT_APPROVED);
    }

    @Transactional
    public void deactivate(UUID customerId, UUID adminId)
    {
        Customer customer = find(customerId);
        requireStatus(customer, AccountStatus.ACTIVE);
        notSelf(customerId, adminId);
        customer.setStatus(AccountStatus.DEACTIVATED);
        auditLogService.record(customerId, AuditAction.ACCOUNT_DEACTIVATED);
    }

    /**
     * Admins never act on their own account. This alone guarantees at least
one active admin always
```

```
     * remains: the last admin has nobody else who could deactivate or delete
them.
     */
    private static void notSelf(UUID targetId, UUID adminId)
    {
        if (targetId.equals(adminId))
            throw new BusinessException(ErrorCode.CANNOT_MODIFY_SELF);
    }

    // reject, reactivate and delete follow the same shape
}
```

The admin's id comes from `AuthenticationUtil.getUserId(authentication)` in the controller, exactly like the customer endpoints. Reuse the existing `deleteAccount` logic for delete, and move `CustomerServiceImpl.deactivateAccount` and `deleteAccount` into this service so the customer side no longer has them.

Return an `AdminCustomerResponse` DTO (id, name, email, provider, status, roles, created date), never the `Customer` entity. The entity has the password hash on it.

Step 4: Frontend

The profile from `GET /users/me` already has a `roles` list, so the SPA knows who is an admin without decoding the token.

1. **`AuthService`**: add `readonly isAdmin = computed(() => this.user()?.roles.includes('ROLE_ADMIN') ?? false);`
2. **`auth.guards.ts`**: add an `adminGuard` that sends signed-out users to `/login` and non-admins to `/products`. Like the other guards, it only improves the experience; the backend is the real lock.
3. **Routes**: one lazy-loaded parent route, so customers never download the admin code:

```
{
  path: 'admin',
  canActivate: [adminGuard],
  loadChildren: () => import('./admin/admin.routes').then((m) =>
m.ADMIN_ROUTES),
},
```

4. **Admin pages** under `src/app/admin/`:
   - `customers`: a table with status tabs (Pending, Active, Deactivated, Rejected) and a count badge on Pending. Each row has the buttons its status allows, from the Step 3 table. Reject and delete open a confirm dialog; reject asks for a reason.

- `customer-detail`: profile, roles, orders and that customer's activity timeline. Reuse the activity component by passing the entries in.

5. **Navbar**: show an "Admin" link when `auth.isAdmin()` is true, with the pending count as a badge.
6. **Sign-up and sign-in screens**:
   - After registering, show a "Thanks, your account is waiting for approval" page instead of signing in.
   - On login errors, map `ACCOUNT_PENDING_APPROVAL`, `ACCOUNT_REJECTED` and `ACCOUNT_DEACTIVATED` to friendly messages.
   - The Google callback page needs the same messages, since `OAuth2LoginFailureHandler` passes the code in the URL.
7. **Account page**: remove the Deactivate and Delete buttons and the danger zone, or replace them with "Contact support to close your account".

Step 5: Audit log and testing

The audit log you already have does the right thing with no extra work. When an admin approves someone, the entry is filed under the **customer's** id, and `@CreatedBy` fills **actor** with the **admin's** email. The customer's Activity page will then show "Account approved · admin@…".

Add these values to `AuditAction` (and to the `AuditAction` type and `ACTIONS` map in the frontend):

- `ACCOUNT_APPROVED`
- `ACCOUNT_REJECTED`, with the reason as details
- `ROLE_GRANTED` and `ROLE_REVOKED`, for when admins can promote users

Test checklist

- [ ] Register by email, then try to sign in: 403 `ACCOUNT_PENDING_APPROVAL`, no token issued
- [ ] Register with Google: lands on the "waiting for approval" message
- [ ] As a customer, call any `/api/v1/admin/**` endpoint: 403
- [ ] As an admin, approve the account: the customer can now sign in
- [ ] Approve the same account again: 409 `ACCOUNT_NOT_PENDING`
- [ ] Deactivate a signed-in customer: their next request with the old token returns 401
- [ ] Admin tries to deactivate or delete themselves: 409 `CANNOT_MODIFY_SELF`
- [ ] With two admins, one can deactivate the other; the remaining admin still cannot deactivate themselves

- [ ] Restart the app with `ADMIN_*` set and an admin already present: no second admin is created
- [ ] Existing customers from before the migration can still sign in
- [ ] The customer's Activity page shows the approval with the admin as actor

The security rules are worth automated tests. A `@WebMvcTest` with `@WithMockUser(roles = "CUSTOMER")` against an admin URL, expecting 403, takes a few lines and guards against someone deleting the `SecurityConfig` rule later.

More ideas for the ADMIN role

The catalogue is the most useful place to start: today products can only be added through the database. Your entities already support most of these ideas (`Product.status` , `stockQuantity` , `Category` , `Brand` , `ProductImage` , `Customer.locked` , `OrderStatus`), so they need endpoints and screens more than new tables.

Idea	What the admin can do	Builds on	Priority
Product management	Create and edit products: name, description, price, currency, category, brand, weight. Hide a product with status `IN_ACTIVE` instead of deleting it, so old orders and carts stay valid	`Product` , `ProductStatus`	Now
Product images	Upload, replace and remove a product's image (check file type and size, at most a few MB)	`ProductImage` , the existing image endpoint	Now
Stock control	Set or adjust stock, see a low-stock list (for example under 5 units), mark out of stock	`stockQuantity` , `OUT_OF_STOCK`	Now
Categories and brands	Add, rename and remove; refuse to remove one that products still use	`Category` , `Brand`	Next

Idea	What the admin can do	Builds on	Priority
Order management	See all orders, filter by status and customer, move an order along allowed steps (Confirmed, In progress, Completed), cancel, handle refund requests	`OrderStatus` incl. `REQUEST_REFUND`	Next
User roles and locks	Promote a customer to admin or remove it (never your own), lock and unlock an account for fraud	`Role`, `Customer.locked`	Next
Store-wide audit log	One screen with every entry, filtered by actor, action and date. Also record catalogue changes ("price 49.99 to 39.99") by adding entity type and id columns to `AUDIT_LOG`	`AuditLog`	Next
Approval emails	Email the customer when they are approved or rejected, so they know when to sign in	Spring Mail	Next
Dashboard	Pending approvals, orders and revenue today, top products, low stock, at a glance	All of the above	Later
Bulk import	Upload a CSV of products; show a preview with row errors before saving anything	Product management	Later
Discounts and coupons	Percentage or fixed codes with an end date and a usage limit, applied at checkout	Checkout (not built yet)	Later
Reports export	Download orders or sales for a date range as CSV	Order management	Later

Keep every write endpoint under `/api/v1/admin/...`, for example `POST /api/v1/admin/products`. Your `SecurityConfig` makes only `GET /api/v1/products/**` public, so a `POST /api/v1/products` would be open to **any signed-in customer** unless

you remember to protect it separately.

Build order and open decisions

Build it in this order, so the app works at every step and you never lock yourself out:

1. Admin seeder (Step 1). Sign in as the admin and check that `roles` includes `ROLE_ADMIN`.
2. Admin endpoints for approve, deactivate, reactivate and delete (Step 3), tested with Swagger.
3. Account status and migration (Step 2). **Do this only after the admin endpoints work, otherwise** new sign-ups pile up with nobody able to approve them.
4. Remove the customer self-service endpoints and update the frontend (Step 4).
5. Product management, then images and stock.

Decisions that are yours to make:

- **What happens to a rejected user?** Keep the row so the email cannot sign up again, or delete it so they can try again later.
- **Can customers still ask to close their account?** GDPR-style laws usually require a way to request deletion. A "Request account deletion" button that creates a task for an admin covers this.
- **Should Google sign-ups need approval too?** This guide says yes, for one consistent rule. You could auto-approve verified emails from trusted domains.
- **Is one ADMIN role enough?** Later you might split it into `ROLE_CATALOG_MANAGER` and `ROLE_SUPPORT`. Keeping all admin URLs under `/api/v1/admin/**` makes that split easy.